Λειτουργικά Συστήματα Άσκηση 1

Ευαγγέλου Ηλίας

Ζώντος Ευστάθιος

Σημείωση

Στο κείμενο που ακολουθεί παρατίθεται μια συνοπτική ανάλυση της φιλοσοφίας σχεδιασμού και των βασικών τεχνικών σημείων για κάθε εργαστηριακή άσκηση. Προκειμένου να αποφευχθούν περιττές επαναλήψεις, στοιχεία που έχουν ήδη επεξηγηθεί διεξοδικά (όπως οι κλήσεις συστήματος της πρώτης άσκησης που χρησιμοποιούνται και στις επόμενες) θα περιγράφονται πιο σύντομα.

Άσκηση 1

Αρχικά υλοποιήθηκε έλεγχος για τη μορφή και το πλήθος των ορισμάτων.

Άνοιγμα αρχείων:

Το πρόγραμμα αρχικά ανοίγει το αρχείο εισόδου χρησιμοποιώντας τη system call open(): open(argv[1], O_RDONLY). Η παράμετρος O_RDONLY δηλώνει ότι το αρχείο θα χρησιμοποιηθεί μόνο για ανάγνωση. Αν η κλήση αποτύχει, εμφανίζεται μήνυμα σφάλματος και το πρόγραμμα τερματίζει.

Στη συνέχεια ανοίγει το αρχείο εξόδου με: open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644).Τα flags σημαίνουν: O_WRONLY : άνοιγμα αρχείου για εγγραφή, O_CREAT : δημιουργία αρχείου αν δεν υπάρχει, O_TRUNC : διαγραφή προηγούμενου περιεχομένου. Η τιμή 0644 καθορίζει τα δικαιώματα πρόσβασης στο αρχείο (σε οκταδικό αριθμό υπό την μορφή user-group-other). Οι συγκεκριμένοι αριθμοί σημαίνουν ότι ο user έχει δικαιώματα read, write, καθώς και ότι όλοι οι υπόλοιπου μόνο read.

Ανάγνωση αρχείων:

Η ανάγνωση του αρχείου πραγματοποιείται με τη system call read(). Τα δεδομένα διαβάζονται σε blocks μεγέθους BUFF_SIZE bytes και αποθηκεύονται στον πίνακα buff. Η επανάληψη πραγματοποιείται με βρόχο while: while ((n = read(fdr, buff, BUFF_SIZE)) != 0). Η μεταβλητή n περιέχει τον αριθμό των bytes που διαβάστηκαν. Για κάθε block δεδομένων που διαβάζεται, το πρόγραμμα ελέγχει όλους τους χαρακτήρες του buffer.

Μέτρηση εμφανίσεων χαρακτήρα:

Για κάθε χαρακτήρα πραγματοποιείται σύγκριση με τον χαρακτήρα που δόθηκε από τον χρήστη. Αν οι χαρακτήρες είναι ίδιοι, αυξάνεται η μεταβλητή cnt. Με τον τρόπο αυτό το πρόγραμμα υπολογίζει το πλήθος των εμφανίσεων του χαρακτήρα σε ολόκληρο το αρχείο.

Δημιουργία μηνύματος αποτελέσματος:

Αφού ολοκληρωθεί η ανάγνωση του αρχείου, δημιουργείται ένα μήνυμα αποτελέσματος που περιλαμβάνει τον χαρακτήρα που αναζητήθηκε, το πλήθος εμφανίσεών του και το όνομα του αρχείου. Για τη δημιουργία του μηνύματος χρησιμοποιείται η συνάρτηση sprintf.

```
sprintf(output, "The character '%c' appears %d times in file %s.\n", c2c, cnt, argv[1]);
```

Εγγραφή Αποτελέσματος:

Για την πιο ασφαλή χρήση του system call write() , υλοποιήθηκε στο utils.c μια βοηθητική συνάρτηση, η write_message(). Η write_message(int fd, const char *message) χρησιμοποιείται για την εγγραφή ενός μηνύματος σε έναν file descriptor. Η ανάγκη για αυτή τη συνάρτηση προκύπτει επειδή η write() δεν εγγυάται ότι θα γράψει όλα τα bytes του buffer σε μία μόνο κλήση. Μπορεί να επιστρέψει μικρότερο πλήθος bytes από το ζητούμενο, δηλαδή να πραγματοποιηθεί partial write.

Για αυτόν τον λόγο, η συνάρτηση:

Υπολογίζει αρχικά το συνολικό μήκος του μηνύματος με strlen(),καλεί επαναληπτικά τη write() για το τμήμα του μηνύματος που απομένει,ενημερώνει κάθε φορά έναν δείκτη θέσης (idx) και σταματά μόνο όταν έχουν γραφτεί όλοι οι χαρακτήρες. Έτσι εξασφαλίζεται ότι όλο το μήνυμα θα σταλεί, ακόμη και αν η write() ολοκληρώσει μόνο μέρος της εγγραφής σε κάποια κλήση. Σε περίπτωση σφάλματος write() επιστρέψει -1 και η συνάρτηση τερματίζει αμέσως.

```
void write_message(int fd, const char *message){
    size_t len = strlen(message);
    ssize_t wcnt;
    size_t idx = 0;
```

```
    do {
        wcnt = write(fd, message + idx, len - idx);
        if (wcnt == -1) {
            return;
        }
        idx += wcnt;
    } while (idx < len);
}
```

Κλείσιμο Αρχείων και Τερματισμός:

Τέλος το πρόγραμμα κλείνει τα αρχεία χρησιμοποιώντας τη system call close(). Η σωστή διαχείριση των αρχείων είναι σημαντική για την απελευθέρωση των πόρων του συστήματος. Μετά το κλείσιμο των αρχείων το πρόγραμμα ολοκληρώνει την εκτέλεσή του.

Δοκιμή προγράμματος:

Φτίαξαμε ένα πρόγραμμα με το παρακάτω μήνυμα προκειμένου να δοκιμάσουμε τον κώδικα μας.

```
1   this is a test file for searching chars.
  1
```

Παρακάτω φαίνονται οι δοκιμές μας.

```
[oslab033@os-node2:~/Ex1s$ ./count_chars hello output i
[oslab033@os-node2:~/Ex1s$ cat output
 The character 'i' appears 4 times in file hello.
[oslab033@os-node2:~/Ex1s$ ./count_chars hello output a
[oslab033@os-node2:~/Ex1s$ cat output
 The character 'a' appears 3 times in file hello.
```

Παρατηρούμε ότι έχουμε την επιθυμητή έξοδο σε κάθε περίπτωση που έχουμε valid input. Υπάρχει η περίπτωση όμως να εισάγουμε invalid input, τότε το πρόγραμμά μας καλύπτει και αυτήν την περίπτωση.

```
oslab033@os-node2:~/Ex1s$ ./count_chars hello output
Please pass the correct number of arguments!
```

Άσκηση 2

1. Η διεργασία παιδί δημιουργείται μέσα από την κλήση συστήματος fork(). Αυτή επιστρέφει στη διεργασία γονέα το PID του παιδιού, αν όλα πάνε καλά, ενώ -1 αν δημιουργηθεί κάποιο σφάλμα. Στο παιδί επιστρέφεται η τιμή 0. Έπειτα με τις κλήσεις συστήματος getpid() και getppid() μπορούμε να εξάγουμε και να εμφανίσουμε από την διεργασία παιδί τα PID του και του γονέα του.

Τέλος, όπως φαίνεται από τον κώδικα, η διεργασία γονέα με την κλήση συστήματος wait() μπλοκάρει και περιμένει το παιδί να ολοκληρώσει την διεργασία του. Στην κλήση συστήματος wait περνάμε ως όρισμα έναν δείκτη σε ακέραιο, στον οποίο θα αποθηκευτούν πληροφορίες (flags και exit status) σχετικά με τον τερματισμό της διεργασίας παιδί. Μετά την ολοκλήρωση εκτυπώνει ένα μήνυμα. Παρακάτω φαίνονται τα αποτελέσματα αυτού του κώδικα.

```
My child's PID is 2291499
Hello world! My PID is 2291499 and my parent's PID is 2291498
My child did it's job
```

Όπως ήταν αναμενόμενο, φαίνεται ότι πρώτα ο γονέας εκτυπώνει το αναγνωριστικό του παιδιού, έπειτα το παιδί εκτυπώνει τα αναγνωριστικά του και του γονέα του και τέλος το παιδί τερματίζει και ο γονέας συνεχίζει με την εκτύπωση του τελευταίου ενημερωτικού μηνύματος.

2. Για αυτή την άσκηση ορίζουμε την μεταβλητή x στον γονέα πριν κληθεί η fork() και της δίνουμε την τιμή 20. Έπειτα αλλάζουμε την τιμή της από την διεργασία παιδί και την

εκτυπώνουμε τόσο από την διεργασία παιδί, όσο και από την διεργασία γονέα. Παρακάτω φαίνεται ο κώδικας:

Το αποτέλεσμα ήταν αυτό:

```
My child's PID is 2292121
Hello world! My PID is 2292121 and my parent's PID is 2292120
The child's variable is 10
My child did it's job
The parent's variable is 20
```

Παρατηρούμε ότι, παρόλο που αλλάξαμε την τιμή της μεταβλητής μας στην διεργασία παιδί η μεταβολή αυτή δεν επηρεάζει τη μεταβλητή στη διεργασία γονέα και αντιστρόφως. Αυτό συμβαίνει γιατί κατα την διάρκεια της κλήσης fork() το ΛΣ δημιουργεί ένα εικονικό αντίγραφο του address space της γονικής διεργασίας για το παιδί. Αν το παιδί πάει να εγγράψει κάτι σε αυτές τις σελίδες της μνήμης ενεργοποιείται ο μηχανισμός Copy On Write και τότε γίνονται τα αντίγραφα φυσικής μνήμης. Επομένως η εγγραφή του παιδιού δεν θα επηρεάσει τις σελίδες μνήμης του γονέα.

3. Σε σχέση με την υλοποίηση της πρώτης άσκησης, η βασική τροποποίηση είναι ότι η καταμέτρηση των εμφανίσεων του ζητούμενου χαρακτήρα δεν πραγματοποιείται πλέον από μία μόνο διεργασία, αλλά ανατίθεται σε μία διεργασία παιδί, η οποία δημιουργείται με τη χρήση της κλήσης συστήματος fork().

Δημιουργείται ένας σωλήνας επικοινωνίας (pipe) μέσω της pipe(pipefd). Ο pipe χρησιμοποιείται ως μηχανισμός επικοινωνίας μεταξύ της διεργασίας παιδιού και της διεργασίας γονέα. Η χρήση του είναι αναγκαία επειδή, μετά τη fork(), οι δύο διεργασίες εκτελούνται σε διαφορετικούς χώρους μνήμης, άρα μία μεταβολή της μεταβλητής cnt στο παιδί δεν είναι ορατή απευθείας στον γονέα.

Με την κλήση fork() δημιουργείται η διεργασία παιδί. Από αυτό το σημείο και μετά υπάρχουν δύο ροές εκτέλεσης:

Διεργασία παιδί

Στο τμήμα if (p == 0) εκτελείται ο κώδικας του παιδιού.
Η διεργασία παιδί κλείνει πρώτα το άκρο ανάγνωσης του pipe (pipefd[0]), γιατί δεν πρόκειται να διαβάσει από αυτόν αλλά μόνο να γράψει.

Στη συνέχεια πραγματοποιεί την κύρια λειτουργία του προγράμματος: διαβάζει το αρχείο εισόδου τμηματικά σε buffer μεγέθους BUFF_SIZE και εξετάζει κάθε byte ξεχωριστά. Για κάθε χαρακτήρα που είναι ίσος με τον ζητούμενο χαρακτήρα c2c, αυξάνει τον μετρητή cnt.

Η επιλογή της ανάγνωσης σε blocks αντί χαρακτήρα-χαρακτήρα είναι πιο αποδοτική, γιατί μειώνει τον αριθμό των system calls.

Όταν ολοκληρωθεί η καταμέτρηση, η διεργασία παιδί στέλνει το αποτέλεσμα στον γονέα γράφοντας την τιμή του cnt στο pipe με τη write_all(pipefd[1], &cnt, sizeof(cnt)). Η συνάρτηση αυτή έχει υλοποιηθεί στο αρχείο utils.c και ουσιαστικά πραγματοποιεί iterative system call write μέχρι να γραφεί όλο το επιθυμητό μήνυμα. Το παιδί **δεν** επιστρέφει το αποτέλεσμα μέσω του exit(), το exit status χρησιμοποιείται μόνο για ένδειξη επιτυχούς ή αποτυχημένου τερματισμού. Αυτό είναι σημαντικό, γιατί το exit status δεν είναι γενικός μηχανισμός μεταφοράς δεδομένων και έχει πολύ περιορισμένη χωρητικότητα(8 bits).

Τέλος, η διεργασία παιδί κλείνει τους descriptors που δεν χρειάζεται πλέον (fdr, fdw, pipefd[1]) και τερματίζει με exit(0) ώστε να δηλώσει επιτυχή ολοκλήρωση.

Διεργασία γονέας

Στο τμήμα else εκτελείται ο κώδικας του γονέα.
Ο γονέας κλείνει το άκρο εγγραφής του pipe (pipefd[1]), αφού θα χρησιμοποιήσει τον pipe μόνο για ανάγνωση.

Στη συνέχεια καλεί wait(&status) ώστε να περιμένει την ολοκλήρωση της διεργασίας παιδιού. Η χρήση της wait()είναι σημαντική για δύο λόγους:

1. εξασφαλίζει ότι ο γονέας δεν θα επιχειρήσει να χρησιμοποιήσει το αποτέλεσμα πριν αυτό παραχθεί,
2. αποτρέπει τη δημιουργία zombie process.

Αμέσως μετά, ο γονέας ελέγχει αν το παιδί τερμάτισε σωστά με τις μακροεντολές WIFEXITED(status) και WEXITSTATUS(status). Αν το παιδί δεν ολοκλήρωσε κανονικά ή επέστρεψε κωδικό λάθους, τότε ο γονέας τερματίζει επίσης αναφέροντας αποτυχία.

Αν όλα έχουν πάει σωστά, ο γονέας διαβάζει από τον pipe την τιμή του μετρητή στη μεταβλητή child_cnt. Ο έλεγχος:

```
read_all(pipefd[0], &child_cnt, sizeof(child_cnt)) != sizeof(child_cnt)
```

είναι ουσιαστικός, γιατί διασφαλίζει ότι διαβάστηκαν ακριβώς τόσα bytes όσα καταλαμβάνει ένας int. Σε διαφορετική περίπτωση το αποτέλεσμα θα μπορούσε να είναι ελλιπές ή μη έγκυρο. Η συνάρτηση read_all() έχει υλοποιηθεί ως wrapper της read στο αρχείο utils για να πραγματοποιεί iterative reading.

Στη συνέχεια, ο γονέας χρησιμοποιεί τη συνάρτηση snprintf() για τη σύνθεση του τελικού μηνύματος, δημιουργώντας με ασφάλεια ένα format string εντός του πίνακα output. Η χρήση της snprintf() προτιμάται έναντι της sprintf() για λόγους προστασίας από υπερχείλιση του buffer, ενώ παράλληλα πραγματοποιείται έλεγχος για τη διασφάλιση της επαρκούς χωρητικότητας του πίνακα.

Τέλος, το μήνυμα καταγράφεται στο αρχείο εξόδου μέσω της συνάρτησης write_all(), η οποία εγγυάται την πλήρη εγγραφή όλων των bytes, ακόμα και στην περίπτωση που η write() επιστρέψει μερικό αποτέλεσμα.

4. Στην άσκηση αυτή υλοποιείται ένα πρόγραμμα το οποίο λειτουργεί ως wrapper για το εκτελέσιμο της πρώτης άσκησης. Συγκεκριμένα, δέχεται ως ορίσματα:

1. το pathname του εκτελέσιμου της Άσκησης 1,
2. το αρχείο εισόδου,
3. το αρχείο εξόδου,

4. τον χαρακτήρα προς αναζήτηση.

Μετά από έλεγχο του πλήθους των ορισμάτων, δημιουργείται μία νέα διεργασία παιδί μέσω της fork(). Η βασική διαφοροποίηση σε σχέση με προηγούμενες ασκήσεις είναι ότι η διεργασία παιδί δεν εκτελεί δικό της κώδικα, αλλά αντικαθιστά πλήρως τον εαυτό της με ένα άλλο πρόγραμμα, χρησιμοποιώντας την κλήση συστήματος execv().

Η execv() δεν δημιουργεί νέα διεργασία.
Αντίθετα, αντικαθιστά πλήρως το image της τρέχουσας διεργασίας με ένα νέο πρόγραμμα.

Πιο συγκεκριμένα:

- ο ίδιος PID παραμένει,
- η μνήμη της διεργασίας αντικαθίσταται,
- ο κώδικας, οι μεταβλητές και η στοίβα της παλιάς διεργασίας χάνονται,
- ξεκινά η εκτέλεση του νέου προγράμματος από την αρχή (main).

Στην παρούσα υλοποίηση, αυτό σημαίνει ότι η διεργασία παιδί μετατρέπεται ουσιαστικά στο πρόγραμμα της Άσκησης 1.

Η execv() έχει τη μορφή:

```
execv(path, argv);
```

όπου:

- path: το pathname του εκτελέσιμου,
- argv: πίνακας από strings που αναπαριστά τα arguments του νέου προγράμματος.

Στον κώδικα:

```
char * newargv[] = {argv[1], argv[2], argv[3], argv[4], NULL};
```

Ο πίνακας newargv κατασκευάζεται έτσι ώστε:

- newargv[0] να είναι το όνομα του εκτελέσιμου (συμβατικά),
- τα επόμενα στοιχεία να είναι τα arguments που περιμένει το πρόγραμμα της Άσκησης 1,
- το τελευταίο στοιχείο να είναι NULL (υποχρεωτικό για τον τερματισμό του πίνακα).

Άσκηση 3

Η βασική ιδέα είναι ότι το αρχείο χωρίζεται σε ίσα τμήματα και κάθε παιδί αναλαμβάνει να επεξεργαστεί μόνο το τμήμα που του αντιστοιχεί. Με αυτόν τον τρόπο η αναζήτηση γίνεται παράλληλα και πιο γρήγορα σε σχέση με μια σειριακή υλοποίηση.

Δημιουργήσαμε μεσα στο utils.c την read_all() που χρησιμοποιείται για την ανάγνωση ακριβώς συγκεκριμένου αριθμού bytes από κάποιο file descriptor, ή μέχρι να συναντηθεί EOF. Αντίστοιχα με τη write(), και η read() μπορεί να επιστρέψει λιγότερα bytes από όσα ζητήθηκαν, χωρίς αυτό να σημαίνει απαραίτητα σφάλμα. Στο πρόγραμμα η read_all() χρησιμοποιείται από τον parent κατά την ανάγνωση των μερικών αποτελεσμάτων από τα pipes. Με αυτόν τον τρόπο διασφαλίζεται ότι θα αναγνωστεί ολόκληρη η τιμή του ακεραίου που έστειλε κάθε child.

Χειρισμός σημάτων:

Το πρόγραμμα χρησιμοποιεί δύο signal handlers: έναν για SIGINT και έναν για SIGCHLD.

Ο handler του SIGINT ενεργοποιείται όταν ο χρήστης πατήσει Ctrl+C. Αντί να τερματίζει αμέσως το πρόγραμμα, ο handler εμφανίζει τον αριθμό των child διεργασιών που θεωρούνται ενεργές εκείνη τη στιγμή. Η πληροφορία αυτή αποθηκεύεται στη μεταβλητή current_children, η οποία έχει δηλωθεί ως volatile sig_atomic_t. Ο συγκεκριμένος τύπος επιλέχθηκε γιατί είναι ο κατάλληλος για μεταβλητές που προσπελαύνονται τόσο από τον κανονικό κώδικα όσο και από signal handlers. Το sig_atomic_t διασφαλίζει ατομική πρόσβαση, ενώ το volatile ενημερώνει τον compiler ότι η μεταβλητή μπορεί να αλλάξει

ασύγχρονα. Μέσα στον signal handler δεν χρησιμοποιούνται συναρτήσεις όπως printf() ή sprintf(), επειδή δεν είναι async-signal-safe. Για τον λόγο αυτό η μετατροπή του αριθμού σε ακολουθία χαρακτήρων γίνεται χειροκίνητα και η έξοδος πραγματοποιείται με την write(), η οποία είναι ασφαλής για χρήση μέσα σε signal handlers.

Ο handler του SIGCHLD καλείται όταν τερματίζει κάποια child διεργασία. Στο εσωτερικό του handler χρησιμοποιείται η waitpid(-1, &status, WNOHANG) μέσα σε βρόχο, ώστε να γίνει συλλογή όλων των terminated children χωρίς μπλοκάρισμα. Κάθε φορά που ένα child συλλέγεται επιτυχώς από τον parent, η μεταβλητή current_children μειώνεται κατά ένα. Με αυτόν τον τρόπο διατηρείται ενημερωμένη η πληροφορία για τον αριθμό των ενεργών child διεργασιών. Όταν ένα child τερματίζει, δεν εξαφανίζεται αμέσως πλήρως από το σύστημα. Παραμένει σε κατάσταση που λέγεται zombie process, μέχρι ο parent να διαβάσει την κατάστασή του με wait() ή waitpid(). Αν ο parent δεν το κάνει αυτό, τα terminated children μένουν zombies.Η κλήση: waitpid(-1, &status, WNOHANG) σημαίνει: -1 = περίμενε οποιοδήποτε child, &status = αποθήκευσε εδώ την κατάσταση εξόδου του child και WNOHANG = μην μπλοκάρεις αν δεν υπάρχει child που να έχει τελειώσει. Η λογική είναι μετά από κάθε επιτυχημένο fork() ο parent κάνει current_children++ και όταν ένα child τελειώνει και το πιάνει ο SIGCHLD handler, γίνεται current_children–. Έτσι, όταν ο χρήστης πατήσει Ctrl+C, ο SIGINT handler μπορεί να εμφανίσει πόσα παιδιά τρέχουν ακόμα.

Υπολογισμός μεγέθους αρχείου:

Πριν ξεκινήσει η παράλληλη επεξεργασία, το πρόγραμμα χρησιμοποιεί τη stat() για να πάρει τα δεδομένα του αρχείου εισόδου και ειδικότερα το μέγεθός του σε bytes. Η τιμή αυτή αποθηκεύεται σε μεταβλητή τύπου off_t. Ο υπολογισμός του μεγέθους είναι απαραίτητος, επειδή το πρόγραμμα πρέπει να χωρίσει το αρχείο σε P επιμέρους περιοχές. Χωρίς γνώση του συνολικού μεγέθους δεν θα ήταν δυνατό να καθοριστούν τα offsets αρχής και τέλους για κάθε child διεργασία.

Δημιουργία pipes:

Για την επικοινωνία ανάμεσα στον parent και στα children δημιουργούνται P pipes, δηλαδή ένα pipe για κάθε child διεργασία. Κάθε pipe χρησιμοποιείται από ένα child για να στείλει στον parent τον αριθμό εμφανίσεων που υπολόγισε. Η χρήση ξεχωριστού pipe για κάθε child απλοποιεί τον σχεδιασμό, διότι αποφεύγεται η ταυτόχρονη εγγραφή πολλών children στο ίδιο κανάλι επικοινωνίας. Έτσι κάθε pipe αντιστοιχεί καθαρά σε μία child διεργασία και ο parent μπορεί να διαβάζει τα αποτελέσματα με προβλέψιμο τρόπο.

Δημιουργία παιδικών διεργασιών:

Το πρόγραμμα δημιουργεί τέσσερις child διεργασίες με επαναληπτικές κλήσεις της fork(). Για κάθε επανάληψη: αν η fork() αποτύχει, εμφανίζεται μήνυμα σφάλματος,αν η τιμή επιστροφής είναι 0, εκτελείται ο κώδικας του child και αν η τιμή είναι θετική, συνεχίζει ο parent. Κάθε child διεργασία εκτελεί ανεξάρτητα την επεξεργασία του τμήματος του αρχείου που της αντιστοιχεί. Ο parent παραμένει υπεύθυνος για τη συλλογή των αποτελεσμάτων και για την τελική εγγραφή στο αρχείο εξόδου. Η χρήση πολλαπλών διεργασιών επιτρέπει την παράλληλη επεξεργασία του ίδιου αρχείου.

Κατανομή του αρχείου σε τμήματα:

Για κάθε child διεργασία υπολογίζονται δύο offsets: start = i * size / P ,end = (i + 1) * size / P όπου i είναι ο δείκτης της child διεργασίας και size το συνολικό μέγεθος του αρχείου. Με βάση τα offsets αυτά, κάθε child αναλαμβάνει ένα τμήμα του αρχείου. Ο διαχωρισμός αυτός γίνεται αριθμητικά με βάση το μέγεθος του αρχείου και εξασφαλίζει ότι τα τμήματα είναι περίπου ίσα. Στη συνέχεια κάθε child καλεί lseek() για να μετακινήσει τον file offset του δικού της file descriptor στη θέση start. Από εκεί και πέρα διαβάζει δεδομένα μέχρι να επεξεργαστεί το κομμάτι του αρχείου που φτάνει έως το end.

Ανάγνωση του αρχείου από κάθε child:

Κάθε child ανοίγει το ίδιο αρχείο εισόδου ξεχωριστά, σε λειτουργία μόνο ανάγνωσης. Αυτό είναι σημαντικό γιατί έτσι κάθε child έχει δικό του file descriptor και ανεξάρτητο file offset. Αν όλα τα παιδιά μοιράζονταν τον ίδιο descriptor, θα υπήρχε πρόβλημα

συγχρονισμού. Η ανάγνωση γίνεται σε τμήματα μεγέθους έως BUFF_SIZE. Για κάθε block που διαβάζεται, το child εξετάζει έναν έναν τους χαρακτήρες και συγκρίνει κάθε byte με τον χαρακτήρα που δόθηκε από τη γραμμή εντολών. Κάθε επιτυχής σύγκριση αυξάνει τον τοπικό μετρητή count. Η διαδικασία συνεχίζεται μέχρι να καλυφθεί όλο το τμήμα του αρχείου που αντιστοιχεί στη συγκεκριμένη child διεργασία. Η χρήση σταθερού buffer κάνει την επεξεργασία αποδοτική, καθώς αποφεύγεται η φόρτωση ολόκληρου του αρχείου στη μνήμη.

Αποστολή των μερικών αποτελεσμάτων:

Αφού κάθε child ολοκληρώσει την καταμέτρηση, γράφει την τιμή του τοπικού μετρητή count στο pipe που της αντιστοιχεί. Μετά την επιτυχημένη αποστολή του αποτελέσματος, το child κλείνει τους descriptors που χρειάστηκε και τερματίζει.

Συλλογή και άθροιση των αποτελεσμάτων:

Αφού δημιουργηθούν όλα τα children, ο parent κλείνει τα write ends των pipes. Στη συνέχεια διαβάζει από κάθε pipe το αποτέλεσμα που έστειλε το αντίστοιχο child και το προσθέτει σε έναν συνολικό μετρητή total. Η συλλογή αυτή υλοποιεί το δεύτερο στάδιο της παράλληλης επεξεργασίας: πρώτα γίνεται ο καταμερισμός του υπολογισμού στα children και στη συνέχεια γίνεται το άθροισμα των επιμέρους αποτελεσμάτων στον parent. Το άθροισμα που προκύπτει αντιστοιχεί στο συνολικό πλήθος εμφανίσεων του ζητούμενου χαρακτήρα σε όλο το αρχείο.

Εγγραφή αποτελέσματος στο αρχείο εξόδου:

Στο τελικό στάδιο το αποτέλεσμα γράφεται στο αρχείο εξόδου, όπως παρουσιάστηκε και στην Άσκηση 1.

Εγκατάσταση signal handlers:

Για τη διαχείριση των σημάτων χρησιμοποιήθηκε η sigaction(), η οποία παρέχει πιο ασφαλή και πιο ελεγχόμενη συμπεριφορά από την signal(). Αρχικά εγκαταστάθηκε handler για το SIGINT, ώστε όταν ο χρήστης πατά Ctrl+C να καλείται η συνάρτηση sighandler. Στη συνέχεια

εγκαταστάθηκε handler για το SIGCHLD, ώστε όταν τερματίζει κάποια child διεργασία να καλείται η sigchld_handler. Και στις δύο περιπτώσεις χρησιμοποιήθηκε το flag SA_RESTART, ώστε system calls που ενδέχεται να διακοπούν από άφιξη σήματος να επανεκκινούνται αυτόματα. Με την sigemptyset() αρχικοποιήθηκε κενή μάσκα σημάτων, ενώ μετά από κάθε κλήση sigaction() έγινε έλεγχος αποτυχίας και κατάλληλος χειρισμός σφάλματος. Τέλος, εδώ χρησιμοποιήθηκε και η sigprocmask() η οποία βοηθά ώστε να μπλοκάρονται προσωρινά νέα SIGCHLD σήματα καθώς γίνεται το fork() και το bookkeeping, ώστε αυτά να γίνονται με ασφάλεια και χωρίς κάποιο race condition.

Ασκηση 4

Protocol:

Το protocol.h αποτελεί την κοινή “γλώσσα” επικοινωνίας όλων των διεργασιών του συστήματος. Εκεί ορίζεται τόσο το msg_type_t, δηλαδή τα είδη των εντολών και απαντήσεων που μπορούν να ανταλλαγούν, όσο και η δομή message_t, η οποία είναι το ενιαίο πακέτο δεδομένων που στέλνεται μέσω των pipes. Με αυτόν τον τρόπο frontend, dispatcher και worker ερμηνεύουν με τον ίδιο ακριβώς τρόπο κάθε μήνυμα. Το πρωτόκολλο που τηρήθηκε στην άσκηση είναι σαφές: ο frontend στέλνει αιτήματα διαχείρισης προς τον dispatcher, ο dispatcher στέλνει εντολές ανάθεσης ή τερματισμού προς τους workers, και οι workers επιστρέφουν αποτελέσματα προς τον dispatcher. Η ύπαρξη κοινού πρωτοκόλλου είναι απαραίτητη ώστε κάθε πλευρά να γνωρίζει ποια πεδία του μηνύματος έχουν νόημα κάθε φορά, για παράδειγμα στα μηνύματα ανάθεσης εργασίας χρησιμοποιούνται τα offset, length, target και job_id, ενώ στα μηνύματα αποτελέσματος χρησιμοποιούνται κυρίως τα pid, count και job_id.

Utils:

Η συνάρτηση parse_user_command() χρησιμοποιείται στον frontend για να μετατρέπει την εντολή που πληκτρολογεί ο χρήστης σε ένα έγκυρο μήνυμα message_t. Αρχικά αφαιρεί τον χαρακτήρα αλλαγής γραμμής και στη συνέχεια ελέγχει αν η εντολή είναι μία από τις επιτρεπτές μορφές, όπως p, i, e, a x ή r x. Αν η εντολή είναι σωστή, συμπληρώνει τα κατάλληλα πεδία της δομής, κυρίως το type και όπου χρειάζεται το value, και επιστρέφει επιτυχία. Αν η σύνταξη είναι λανθασμένη, επιστρέφει αποτυχία, ώστε ο frontend να ενημερώσει τον χρήστη ότι έδωσε μη έγκυρη εντολή. Με αυτόν τον τρόπο συγκεντρώνεται όλη η λογική ερμηνείας των εντολών σε ένα μόνο σημείο.

Η read_until() είναι βοηθητική συνάρτηση για ανάγνωση χαρακτήρων από file descriptor μέχρι να εμφανιστεί ένας συγκεκριμένος delimiter, συνήθως το \n, ή μέχρι να γεμίσει ο buffer. Στην

άσκηση χρησιμοποιείται τόσο για την ανάγνωση εντολών από το standard input όσο και για την ανάγνωση γραμμών κειμένου από το pipe απαντήσεων του dispatcher. Η συνάρτηση διαβάζει προσεκτικά χαρακτήρα-χαρακτήρα, χειρίζεται την περίπτωση διακοπής από σήμα με EINTR και στο τέλος τοποθετεί πάντα τον τερματικό χαρακτήρα \0, ώστε το αποτέλεσμα να είναι έγκυρο C string. Έτσι εξασφαλίζεται ασφαλής και ελεγχόμενη γραμμική ανάγνωση χωρίς υπερχείλιση του buffer.

Frontend:
Η frontend διεργασία ξεκινά ελέγχοντας τα ορίσματα της γραμμής εντολών, δηλαδή το αρχείο εισόδου και τον χαρακτήρα προς αναζήτηση. Στη συνέχεια δημιουργεί δύο pipes: ένα για την αποστολή εντολών προς τον dispatcher και ένα για τη λήψη απαντήσεων από αυτόν. Έπειτα καλεί fork() και στο παιδί εκτελείται με execv() το πρόγραμμα dispatcher, στο οποίο περνιούνται ως ορίσματα τα κατάλληλα file descriptors μαζί με το όνομα του αρχείου και τον χαρακτήρα στόχο.
Η frontend παραμένει σε βρόχο και χρησιμοποιεί select() ώστε να παρακολουθεί ταυτόχρονα το standard input και το pipe απαντήσεων από τον dispatcher. Με αυτόν τον τρόπο μπορεί είτε να διαβάζει εντολές του χρήστη είτε να εμφανίζει ενημερώσεις κατάστασης από τον dispatcher χωρίς να μπλοκάρει άσκοπα. Οι εντολές του χρήστη μετατρέπονται σε message_t μέσω της parse_user_command(). Για τις εντολές progress και info δεν στέλνεται ολόκληρο μήνυμα από pipe, αλλά χρησιμοποιούνται τα σήματα SIGUSR1 και SIGUSR2 αντίστοιχα, ώστε ο dispatcher να απαντά άμεσα με την τρέχουσα κατάστασή του.

Dispatcher:
Ο dispatcher είναι το βασικό συντονιστικό κομμάτι του συστήματος. Αρχικά εγκαθιστά signal handlers για SIGUSR1, SIGUSR2, SIGCHLD, SIGINT και SIGTERM. Τα δύο πρώτα χρησιμοποιούνται για αιτήματα πληροφόρησης από το frontend, το SIGCHLD για την ανίχνευση τερματισμού worker, ενώ τα SIGINT και SIGTERM οδηγούν σε ελεγχόμενο τερματισμό του συστήματος.
Στη συνέχεια ο dispatcher διαβάζει το μέγεθος του αρχείου με stat() και το χωρίζει σε σταθερά chunks μεγέθους CHUNK_SIZE. Για κάθε chunk δημιουργείται ένα job, το οποίο έχει μοναδικό job_id, offset, length και κατάσταση εκτέλεσης. Έτσι το αρχείο δεν αντιμετωπίζεται ως ένα ενιαίο αντικείμενο αλλά ως σύνολο ανεξάρτητων εργασιών που μπορούν να ανατεθούν δυναμικά σε workers.
Κατά την εκκίνηση δημιουργούνται αρχικά δύο workers. Ο dispatcher διατηρεί πίνακα με τα στοιχεία κάθε worker, όπως PID, file descriptors επικοινωνίας, αν είναι ενεργός, αν είναι

απασχολημένος και ποιο job εκτελεί. Η ανάθεση εργασιών γίνεται με round-robin λογική στους διαθέσιμους workers. Όταν υπάρχει pending job και ελεύθερος worker, ο dispatcher δημιουργεί μήνυμα τύπου CMD_ASSIGN_WORK και του στέλνει το αντίστοιχο τμήμα του αρχείου.

Worker:

Κάθε worker εκτελείται ως ξεχωριστό πρόγραμμα μέσω execv(). Ανοίγει το αρχείο εισόδου μόνο για ανάγνωση και στη συνέχεια περιμένει εντολές από τον dispatcher. Όταν λάβει μήνυμα CMD_ASSIGN_WORK, χρησιμοποιεί τα πεδία offset και length για να προσπελάσει ακριβώς το κομμάτι του αρχείου που του ανατέθηκε. Η ανάγνωση γίνεται με pread(), κάτι που είναι ιδιαίτερα χρήσιμο γιατί επιτρέπει τυχαία προσπέλαση σε συγκεκριμένες θέσεις του αρχείου χωρίς να εξαρτάται η λογική από κοινό file offset.

Ο worker μετρά το πλήθος εμφανίσεων του χαρακτήρα target στο συγκεκριμένο chunk και στη συνέχεια στέλνει πίσω στον dispatcher ένα μήνυμα CMD_WORK_RESULT που περιέχει το PID του, το job_id και το τοπικό αποτέλεσμα count. Αν λάβει μήνυμα CMD_TERMINATE_WORKER, τερματίζει καθαρά κλείνοντας τα file descriptors του.

Συλλογή αποτελεσμάτων και πρόοδος:

Ο dispatcher παρακολουθεί τα pipes επιστροφής όλων των ενεργών workers επίσης με select(). Όταν κάποιος worker ολοκληρώσει την εργασία του, ο dispatcher διαβάζει το αποτέλεσμα, ενημερώνει την κατάσταση του αντίστοιχου job σε JOB_DONE και προσθέτει το μερικό αποτέλεσμα στον συνολικό υπολογισμό. Παράλληλα ενημερώνει ότι ο worker είναι πάλι διαθέσιμος για νέα εργασία.

Για την εντολή progress υπολογίζεται το ποσοστό ολοκλήρωσης ως ο λόγος των jobs που έχουν τελειώσει προς το συνολικό πλήθος jobs. Μαζί εμφανίζεται και το πλήθος των matches που έχουν ήδη βρεθεί. Για την εντολή info εμφανίζεται το πλήθος των ενεργών workers καθώς και πληροφορίες για το αν κάποιος worker είναι busy και ποιο job επεξεργάζεται εκείνη τη στιγμή.

Δυναμική προσθήκη και αφαίρεση workers:

Ένα σημαντικό χαρακτηριστικό της άσκησης είναι ότι το worker pool μεταβάλλεται δυναμικά κατά την εκτέλεση. Με την εντολή add ο dispatcher δημιουργεί νέους workers και τους εντάσσει άμεσα στο σύστημα. Με την εντολή remove αποστέλλεται μήνυμα τερματισμού σε υπάρχοντες workers, ώστε η μείωση να γίνεται ελεγχόμενα και όχι βίαια. Έτσι το σύστημα μπορεί να προσαρμόζει τον βαθμό παραλληλισμού του κατά τη διάρκεια της λειτουργίας του.

Ανθεκτικότητα σε αποτυχία worker:

Η ύπαρξη του SIGCHLD handler επιτρέπει στον dispatcher να ανιχνεύει αν κάποιος worker τερματίστηκε απρόσμενα. Σε αυτή την περίπτωση, αν ο worker είχε job σε κατάσταση

JOB_IN_PROGRESS, το job δεν χάνεται αλλά επιστρέφει στην κατάσταση JOB_PENDING, ώστε να μπορεί να ανατεθεί ξανά σε άλλον διαθέσιμο worker. Αυτή η λογική είναι ο βασικός μηχανισμός fault tolerance της υλοποίησης, γιατί εξασφαλίζει ότι η ολοκλήρωση του συστήματος δεν εξαρτάται από την επιβίωση κάθε μεμονωμένου worker.

Τερματισμός:

Όταν ολοκληρωθούν όλα τα jobs, ο dispatcher υπολογίζει το τελικό άθροισμα των εμφανίσεων του χαρακτήρα και το στέλνει στο frontend ως FINAL RESULT. Σε περίπτωση εντολής shutdown ή λήψης σήματος τερματισμού, ο dispatcher στέλνει CMD_TERMINATE_WORKER σε όλους τους ενεργούς workers, κλείνει τα pipes και περιμένει με waitpid() να τερματίσουν. Με αυτόν τον τρόπο αποφεύγονται orphan ή zombie διεργασίες και το σύστημα κλείνει καθαρά.